DATA 260 - Homework 7

Bhushan Waingankar | SID4 7085

Repository: https://github.com/shoddy16/data260-7085

Configuration: SID4=7085; PORT_BASE=8785; PREFIX=s7085; SEED=7085; VERIFY_SEED=267085; DOMAIN_ID=5

Hardware: Windows 10; Intel Core i5-6200U at 2.30 GHz; 4 logical CPUs; approximately 7.8 GB RAM (recorded in the existing HW5/HW6 report artifacts).

Local model: Ollama llama3.2:3b. Validated implementation/evidence commit: 85d98f4. A separate packaging commit will carry this metadata before the hw7 tag is created. The existing hw6 tag remains unchanged.

Implementation, verification, evidence, and limitations

This report separates deterministic offline evidence, live local FastAPI/browser evidence, and checks that were not possible because MySQL, MongoDB, Ollama, repository-wide lint, or final Git submission was unavailable.

1. Source materials and requirements

The official DATA260_HW7_final.docx was read completely: seven pages, all paragraphs and three tables. DATA236_DEMO7.zip was readable: 6,943 entries and 31,983,746 bytes. Inspected reference material included its README, Ruff/pre-commit configuration, FastAPI JWT/RBAC routes, fake database, tests, and CI workflow. The ZIP is a generic reference and does not contain the HW1/HW5 agent, Reflexion, bike-share, prompt-injection, or file-tool implementation.

2. Implementation and code snippets

Part 1 - JWT and role authorization

```
# jwt_auth.py
password_hash = hash_password(password)
user = authenticate_user(db, username, password)
return create_access_token(user.username, user.role)

# route policy
reader: list/read
writer or moderator: create/update
moderator: delete
```

The existing domain routes use bearer authentication. Automated API tests cover successful and failed login, missing authentication, reader denial, writer creation, authenticated listing, writer delete denial, and moderator delete success. The migration source creates the MySQL users fields and stores hashed passwords, but a live MySQL migration was not run.

Part 2 - Handwritten agent loop

```
result = run_agent(user_input, model=model, tool_executor=execute_tool)
# each tool result is appended with role=tool
# max_steps and max_repairs bound execution
```

The loop validates search, detail_lookup, and aggregate actions, retries one malformed action, records tool calls, and returns a bounded stop reason. The captured deterministic evidence covers A, B, C, and nonexistent-ID recovery; no live Ollama call is claimed.

Part 3 - Bounded Reflexion

```
passed, reason = verify_task(task, result)
if not passed: store concise evidence-backed reflection and retry
max_retries = 2; stop_reason in {success, human_review, budget_exhausted}
```

The verifier accepts the final A/B/C deterministic results. Unsupported-claim counts and token counts are unavailable because the scripted model exposes no token accounting and no claim extractor was implemented. False verifier rejections were 0/3 final results; the first intentionally incomplete attempts were rejected before successful retries.

Part 4 - Citi Bike MRKL tools

```
csv_sql(sql) -> read-only SELECT over uploaded CSV
policy_retriever(url, query, k) -> short cited passages
calculator(expression) -> whitelisted arithmetic only
common result shape: success, data, error, source, ts
```

The UI accepts one CSV and pricing URL, shows the tool timeline, cost comparison, decision, citation, step count, and stop reason. The two deterministic cases use 600 and 500 rows and the official pricing snapshot hash.

Part 5 - Prompt injection

```
retrieved pricing text is labeled untrusted data
csv_sql accepts one read-only SELECT statement regardless of model text
raw CSV rows are not included in the final recommendation
```

The local hostile pricing fixture exercises recommendation override, non-read-only SQL, and CSV-content inclusion. Full attack traces are in the raw JSON artifact.

Part 6 - File tools and safety gate

```
read_file, write_file, edit_file, glob_files, search_in_files, bash
SAFE_PREFIXES allow-list plus independent is_destructive deny-list
confirm_bash(command, auto_approve=False) -> allow/block and reason
```

The integration test creates Python files, performs glob -> search -> read, and the safety test checks five allowed and five blocked commands. The final raw result contains no false allows in that deterministic set.

3. Verification and metrics

Check	Actual result
HW7/API/HW6 regression tests	11 passed in the combined pytest run; later split runs recorded 5 and 5 due the repository test-run environment.
Targeted Ruff	Passed on HW7/API files; exit 0.
Targeted pre-commit	Passed Ruff check and Ruff format on HW7 files; exit 0.
Compileall	Passed: python -m compileall -q .
Deterministic evidence	Six checks passed: two scenarios, differing decisions, 20 attempts, injection artifact, file integration, and 5/5 safety gate.
Live local app	GET /hw7 and the 500-row API scenario returned HTTP 200.
Unavailable checks	Live MySQL, MongoDB, Ollama, full-repository Ruff/pre-commit, and final Git submission were not completed.

Bike-share metrics

Scenario	Rows	Decision	Pay/use	Membership + residual	Steps/stop
membership_wins	600	Buy Monthly Membership	$7,480	$2,669	4 / success
pay_per_use_wins	500	Pay Per Ride/Minute	$100	$239	4 / success

Reliability: 20/20 deterministic calls were well formed before and after repair; classification counts were none=20 and all other failure classes=0. Token counts are unavailable, not zero. Unsupported-claim metrics are unavailable because no claim ledger exists. Pricing snapshot SHA-256: 8a1c1952ab94c6a3820c3d7ee2aac7c9b5d9fd6ebc52d69937e2996785b5a9de.

4. Required evidence screenshots

Bhushan Waingankar — HW7 live API evidence

Responses from the running FastAPI application at 127.0.0.1:8785


```
successful login (200): HTTP 200
JWT returned

failed login (401): HTTP 401
{"detail":"Invalid username or password"}

missing token /auth/me (401): HTTP 401
{"detail":"Bearer token required"}

invalid token /auth/me (401): HTTP 401
{"detail":"Invalid or expired bearer token"}

reader create (403): HTTP 403
{"detail":"Insufficient role for this operation"}

writer create (201): HTTP 201
{"restaurantName":"Live HW7 Evidence Cafe","location":"San
Jose","email":"evidence@example.com","description":"evidence","category":"routine","id":1,"restaurant_id":1,"inspection_code":"s7085-i-
e17f3762","score":100,"created_at":"2026-10-26T02:08:11.119371","updated_at":"2026-10-26T02:08:11.119371","restaurant":{"name":"Live HW7
Evidence Cafe","location":"San Jose","id":1,"code":"s7085-r-bd316780","created_at":"2026-10-26T02:00:45.076416","updated_at":"2026-10-
26T02:00:45.076416"}}

authenticated list (200): HTTP 200
[{"restaurantName":"Live HW7 Evidence Cafe","location":"San
Jose","email":"evidence@example.com","description":"evidence","category":"routine","id":1,"restaurant_id":1,"inspection_code":"s7085-i-
e17f3762","score":100,"created_at":"2026-10-26T02:08:11.119371","updated_at":"2026-10-26T02:08:11.119371","restaurant":{"name":"Live HW7
Evidence Cafe","location":"San Jose","id":1,"code":"s7085-r-bd316780","created_at":"2026-10-26T02:00:45.076416","updated_at":"2026-10-
26T02:00:45.076416"}}]

writer delete (403): HTTP 403
{"detail":"Insufficient role for this operation"}

moderator delete (204): HTTP 204
```


Figure 1. Live browser evidence from the running FastAPI application: successful/failed login, missing and invalid token, role-denied create/delete, successful writer create, authenticated list, and moderator delete.

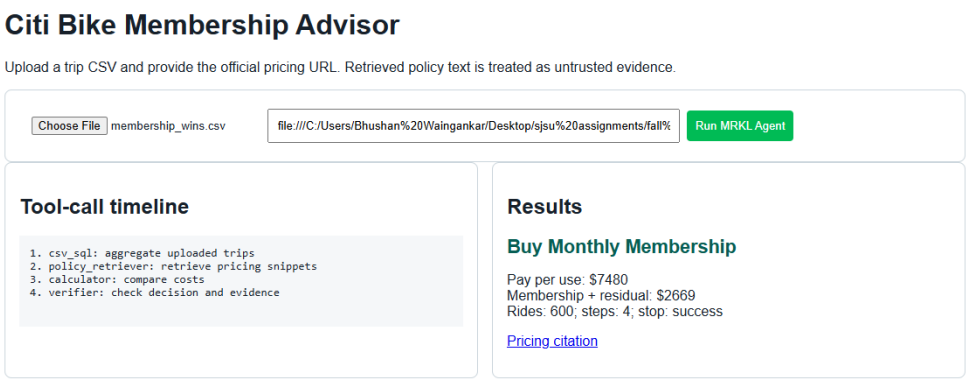


Figure 2. Live Citi Bike UI membership-winning case: four-step timeline, decision, cost comparison, 600 rides, citation, and success stop reason.

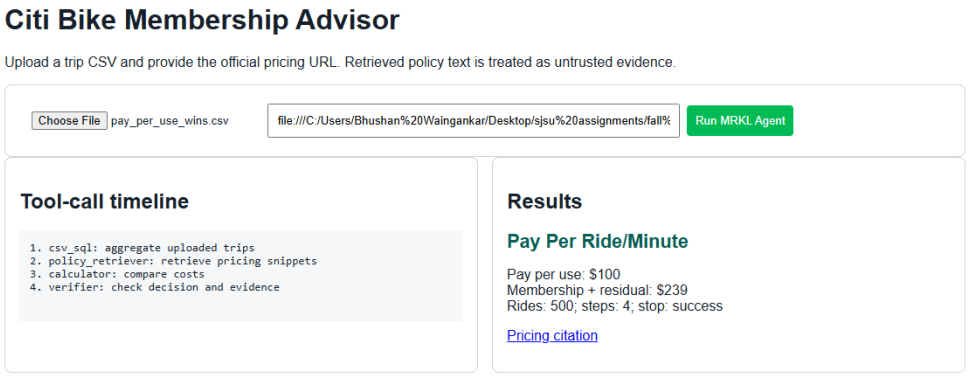


Figure 3. Live Citi Bike UI pay-per-use case: four-step timeline, decision, cost comparison, 500 rides, citation, and success stop reason.

Bhushan Waingankar — HW7 Part 2 evidence

Actual run_agent outputs using the fixed deterministic verification cases.


```
{
  "A search then detail": {
    "stop_reason": "success",
    "tool_call_count": 2,
    "answer": "Task A complete"
  },
  "B aggregate then supporting detail": {
    "stop_reason": "success",
    "tool_call_count": 2,
    "answer": "Task B complete"
  },
  "C search then two details": {
    "stop_reason": "success",
    "tool_call_count": 3,
    "answer": "Task C complete"
  },
  "nonexistent ID recovery": {
    "stop_reason": "success",
    "tool_call_count": 1,
    "answer": "No inspection found; handled without crashing"
  }
}
```


Figure 4. Deterministic run_agent evidence for tasks A, B, C, and nonexistent-ID recovery.

Bhushan Waingankar — HW7 Part 6 evidence

Actual file-tool integration and deterministic safety-gate outputs.


```
{
  "integration": {
    "glob": [
      "C:\\Users\\Bhushan Waingankar\\Desktop\\sjsu assignments\\fall 2026\\Agentic ai 260\\Homeworks\\assignment 1\\data260-
7085\\tmp\\hw7_screenshot_sandbox\\one.py",
      "C:\\Users\\Bhushan Waingankar\\Desktop\\sjsu assignments\\fall 2026\\Agentic ai 260\\Homeworks\\assignment 1\\data260-
7085\\tmp\\hw7_screenshot_sandbox\\two.py"
    ],
    "search": [
      {
        "path": "C:\\Users\\Bhushan Waingankar\\Desktop\\sjsu assignments\\fall 2026\\Agentic ai 260\\Homeworks\\assignment 1\\data260-
7085\\tmp\\hw7_screenshot_sandbox\\one.py",
        "line": 1,
        "text": "DEBUG = True"
      }
    ],
    "read": "DEBUG = True"
  },
  "safety_gate": [
    {
      "command": "python -c \"print(1)\"",
      "allowed": true
    },
    {
      "command": "dir",
      "allowed": true
    },
    {
      "command": "type file.txt",
      "allowed": true
    },
    {
      "command": "rg DEBUG .",
      "allowed": true
    },
    {
      "command": "python --version",
      "allowed": true
    },
    {
      "command": "rm -rf sandbox",
      "allowed": false
    },
    {
      "command": "sudo chmod 777 file",
      "allowed": false
    },
    {
      "command": "format C:",
      "allowed": false
    },
    {
      "command": "dd if=x of=/dev/sda",
      "allowed": false
    },
    {
      "command": "echo x > /etc/x",
      "allowed": false
    }
  ]
}
```


Figure 5. Deterministic file-tool integration and ten-command safety-gate output.

5. Raw artifacts and command record

The following artifacts are included under reports/hw07/raw/: agent_reliability_20.csv, original_vs_reflexion.json, bike_share_scenarios.json, membership_wins.csv, pay_per_use_wins.csv, citi_bike_pricing_snapshot.txt, hostile_pricing_fixture.html, prompt_injection_before_after.json, and file_tools_and_safety.json.

RUN_LOG.txt records deterministic evidence generation, targeted tests, lint/pre-commit checks, compileall, and live local browser capture. METRICS.md records measured values and explicitly labels unavailable token and unsupported-claim metrics. AI_USE.md answers the four required disclosure questions. verification.json contains SID4, seeds, model, checks, and the validated implementation commit 85d98f4.

6. Limitations and submission sequencing

The report records the validated implementation/evidence commit 85d98f4. The final packaging commit contains only the verified commit metadata and regenerated report; the hw7 tag will point to that packaging commit. This two-commit sequence avoids claiming that a commit contains its own unknowable hash. Live MySQL, MongoDB, and Ollama/model verification remains unavailable. The professor-required lint-failure-before-commit terminal screenshot remains unavailable because native terminal screenshot capture is unavailable.

After the packaging commit, Git verification must confirm that hw7 points to the packaging commit and that origin/hw6 still points to 0307ac55b1ec9d6e48561a43e5671e21c09fd708.

Evidence manifest

Screenshots and supporting files are enumerated in reports/hw07/EVIDENCE_MANIFEST.md. The report does not claim that unavailable live services or unperformed commit/tag actions succeeded.